Topic 5: Input and Output — cin and cout

4.1 Output with cout

- `cout << "Hello, World!" << endl;` — prints text to the console
- `<<` is the insertion operator — chain multiple outputs: `cout << "Name: " << name << endl;`
- `endl` flushes the buffer and moves to a new line; `"\n"` just moves to new line (faster)
- Print variables: `cout << "Score: " << score << endl;`

4.2 Input with cin

- `cin >> age;` — reads input from the user and stores it in `age`
- `>>` is the extraction operator
- Multiple inputs: `cin >> a >> b;` reads two values separated by space or Enter
- cin skips leading whitespace automatically
- cin fails with spaces in strings — use `getline()` instead

4.3 Reading Strings with getline

- `cin >> name;` reads only one word — stops at the first space
- `getline(cin, name);` reads the entire line including spaces
- After using `cin >>` , call `cin.ignore();` before `getline()` to discard the leftover newline

4.4 Formatting Output with iomanip

- `#include <iomanip>` to access formatting tools
- `setw(10)` : set minimum field width — right-aligns by default
- `setprecision(2)` with `fixed` : show exactly 2 decimal places — `fixed << setprecision(2)`
- Example: `cout << fixed << setprecision(2) << 3.14159;` → `3.14`

4.5 endl vs "\n"

- Both move to the next line
- `endl` also flushes the output buffer — slower but ensures output appears immediately
- `"\n"` is faster — preferred in loops and performance-sensitive code

- For simple programs the difference is negligible
